CipherRescue

Open-Source Full-Disk Encryption Recovery Framework

SCPR-Based Failure Diagnosis in Physical FDE Systems

Document Type: Comprehensive Technical Specification & Research Document

Covers: Mathematical Foundations · Edge Cases & Solutions · Systematic Literature Review · Architecture · Publication Roadmap

Status: Pre-Publication Draft · Confidential

Licence: GPL-3.0 (code) · CC-BY-4.0 (documentation)

Version 2.0 · 2025

Abstract

CipherRescue is a research-grade, open-source forensic recovery framework for physical full-disk encrypted storage systems. It addresses a clearly documented gap in the tool landscape: no existing open-source tool combines automated multi-scheme detection, guided recovery orchestration, mandatory safety-gated backup, and evidence-grade diagnostic reporting in a single bootable environment.

The primary theoretical contribution is the application of Set Covering Problems with Reasons (SCPR) to failure diagnosis in cryptographic storage systems — constituting the *third* domain application of SCPR, following its genesis in Satisfiability Modulo Theories solving (Sadeghimanesh and England, 2022) and its application to Functional Enrichment Analysis in bioinformatics (Babatunde, 2025a). Failure signals observable on a device (SMART attributes, entropy measurements, header anomalies) are modelled as the universe of a covering instance. Failure mode hypotheses constitute the covering sets. The optimal cover — computed via LP relaxation and Beasley's reduction process rather than greedy approximation — produces a parsimonious, reason-annotated diagnosis where LP dual variables serve as evidence weights. The SCPR formulation and generalised Beasley reduction follow directly from the primary SCPR lineage (Babatunde et al., 2026).

This document provides the complete mathematical specification of all seven architectural layers, formal proofs of key safety and correctness properties, a comprehensive edge-case analysis with proposed solutions, a systematic literature review following PRISMA 2020 guidelines situating CipherRescue within the established SCPR publication record, and a structured publication roadmap targeting two venues.

Contents

Abstract **1**

I Mathematical Foundations **7**

1 Notation and Conventions **8**

2 Layer 1 — Boot Environment: Formal Specification **9**
2.1 System Model 9
2.2 Reproducible Build Guarantee 10

3 Layer 2 — Detection Engine **11**
3.1 Device Graph 11
3.2 Scheme Detection — Signature Matching 11
3.3 VeraCrypt Detection via Entropy Analysis 12
3.4 Health Scoring 12

4 Layer 3 — Safety and Audit Layer **14**
4.1 Backup Token and Integrity 14
4.2 Hash-Chained Audit Log 14
4.3 WriteBlocker Formal Contract 15

5 Layer 4 — Plugin Layer **16**
5.1 Formal Plugin Contract 16

6 Layer 5 — SCPR-Based Diagnostic Engine **17**
6.1 Classical Set Cover — Recap 17
6.2 Set Covering Problems with Reasons (SCPR) 17
6.3 CipherRescue SCPR Instantiation 18
6.4 LP Formulation 18
6.5 Beasley Reduction and Variable Fixing 19

7 Layer 6 — Orchestration Engine **21**
7.1 Workflow State Machine 21

8 Layer 7 — User Interface **22**
8.1 Progressive Disclosure 22

II Edge Cases and Solutions 23

9 Hardware and Architecture Edge Cases 24
9.1 Multi-Scheme Devices . . . 24
9.2 NVMe Namespace Encryption . . . 24
9.3 RAID Arrays with FDE . . . 25
9.4 Partial Encryption Interruption . . . 25
9.5 Degraded NAND Flash and Retry Strategy . . . 25

10 Security Edge Cases 27
10.1 ISO Verification UX Gap . . . 27
10.2 Memory Forensics Window . . . 27
10.3 Plugin Supply Chain Attack . . . 28
10.4 Adversarial Signal Injection . . . 28

11 Forensic and Legal Edge Cases 29
11.1 Chain of Custody . . . 29
11.2 Jurisdiction-Specific Legal Constraints . . . 29
11.3 Filesystem Timestamp Contamination . . . 30
11.4 Clock Accuracy in Air-Gapped Environments . . . 30

12 Enterprise and Operational Edge Cases 31
12.1 Headless Server Recovery . . . 31
12.2 Key Escrow Integration . . . 31
12.3 Partial Backup on Insufficient Destination Space . . . 32
12.4 Concept Drift in Training Data . . . 32
12.5 Multi-Device SCPR Correlation . . . 33
12.6 User-Facing Recoverability Assessment . . . 33

III Systematic Literature Review 34

13 SLR Protocol 35

14 RQ1 — Full-Disk Encryption Recovery 36
14.1 Theme A — Encryption Scheme Specifications . . . 36
14.2 Theme B — Disk Forensics and Recovery . . . 36
14.3 Theme C — Automated Failure Detection in Storage . . . 36
14.4 Theme D — Tool Landscape Gap Analysis . . . 37

15 RQ2 — SCPR Theoretical Lineage 38
15.1 Preamble: The Primary SCPR Lineage . . . 38
15.2 Theme E — Set Cover Foundations . . . 39
15.3 Theme F — Model-Based Diagnosis . . . 39
15.4 Theme G — SCPR-Adjacent Formalisms . . . 40
15.5 Theme H — SCPR in Bioinformatics . . . 40
15.6 SLR Findings Summary . . . 41

IV Publication Roadmap and Project Governance 42

16 Publication Strategy 43
16.1 Situating CipherRescue within the Existing Publication Record . . . 43

16.2 CipherRescue Publication Roadmap 43

17 Open-Source Governance and Intellectual Property **45**
17.1 Licensing Strategy 45
17.2 Competitive Durability 45

V References **47**

List of Tables

3.1 Encryption scheme detection signatures 12

6.1 Failure mode taxonomy 18

12.1 Recoverability classification for non-expert display 33

13.1 SLR search parameters 35

14.1 Foundational encryption scheme literature 36
14.2 Disk forensics literature 36
14.3 Storage failure detection literature 37
14.4 Gap analysis: existing tools vs. CipherRescue 37

15.1 Primary SCPR lineage — works that constitute SCPR as a field 38
15.2 Set cover foundational literature 39
15.3 Model-based diagnosis literature and relationship to SCPR 40
15.4 SCPR-adjacent formalisms 40
15.5 Bioinformatics FEA literature and structural parallel 40

16.1 CipherRescue publication roadmap 43

List of Figures

Part I

Mathematical Foundations

Chapter 1

Notation and Conventions

Throughout this document we use the following conventions.

- Calligraphic letters $(\mathcal{U}, \mathcal{S}, \mathcal{F}, \mathcal{R}, \mathcal{D})$ denote sets or spaces.
- Lowercase bold $(\mathbf{x}, \mathbf{y}, \mathbf{w})$ denote vectors.
- Uppercase Roman (H, G, M) denote functions or structures.
- $\models$ denotes logical satisfaction; $\vdash$ denotes derivability.
- log without a base denotes the natural logarithm unless stated.
- $[n] = \{1, \ldots, n\}$ for any positive integer n.
- $\mathbb{1}[\cdot]$ is the indicator function.
- $H_n = \sum_{k=1}^{n} \frac{1}{k}$ is the n-th harmonic number.

Chapter 2

Layer 1 — Boot Environment: Formal Specification

2.1 System Model

We model the boot environment as a minimal trusted computing base (TCB). Let the system state space be Ω, where each $\omega \in \Omega$ is a complete description of hardware registers, loaded code, and memory contents at a given instant.

Definition 2.1: Boot State

A *boot state* $b \in B \subseteq \Omega$ is a state satisfying:

(i) Firmware control has transferred to the CipherRescue bootloader;
(ii) No writable mount-points exist on target block devices;
(iii) The Python runtime and all bundled tools are integrity-verified.

Definition 2.2: Integrity Verification

Let $H_{\text{SHA}} : \{0,1\}^* \to \{0,1\}^{256}$ denote SHA-256. The live image is accepted if and only if $H_{\text{SHA}}(\text{image}) = H_{\text{published}}$, where $H_{\text{published}}$ is the value recorded in the GPG-signed release manifest.

Under the random oracle model, finding $\text{image}' \neq \text{image}$ such that $H_{\text{SHA}}(\text{image}') = H_{\text{SHA}}(\text{image})$ requires 2^{128} operations (birthday bound). This provides a computationally sound integrity guarantee.

For all $\omega \in B$ and all target devices $d \in D$:

$$\texttt{mounted_rw}(d, \omega) = \texttt{false}.$$

Proof. By construction, the Alpine `initramfs` mounts only the in-memory `tmpfs` as writable. All block devices are enumerated via `lsblk` with no mount action. The Write-Blocker (Layer 3) independently enforces the same constraint at the application level. A violation therefore requires simultaneous compromise of both the OS-level mount policy and the application-level gate — which lies outside the defined threat model. □

2.2 Reproducible Build Guarantee

The live image is built via a Docker-based Alpine customisation pipeline. Let $\mathcal{B} : \text{Source} \to \text{ISO}$ be the build function.

Definition 2.3: Reproducible Build

A build function $\mathcal{B}$ is *reproducible* if for all source inputs s:

$$\mathcal{B}(s) = \mathcal{B}(s) \quad \text{(byte-for-byte identical on repeated invocations).}$$

This requires pinned package versions, deterministic filesystem ordering, and no embedded timestamps. CipherRescue achieves this via `SOURCE_DATE_EPOCH` propagation and `apk` package hash verification.

Chapter 3

Layer 2 — Detection Engine

3.1 Device Graph

Definition 3.1: Device Graph

$G = (V, E)$ where:

$$\begin{aligned} V &= V_{\text{phys}} \cup V_{\text{part}} && \text{(physical drives} \cup \text{logical partitions)} \\ E &\subseteq V_{\text{phys}} \times V_{\text{part}} && \text{(containment edges)} \end{aligned}$$

G is a directed acyclic graph (DAG).

Proof. $E \subseteq V_{\text{phys}} \times V_{\text{part}}$ and $V_{\text{phys}} \cap V_{\text{part}} = \emptyset$. A cycle would require at least one edge $(v_{\text{part}}, v_{\text{phys}}) \in E$, contradicting $E \subseteq V_{\text{phys}} \times V_{\text{part}}$. Therefore G is acyclic by type separation. □

3.2 Scheme Detection — Signature Matching

Definition 3.2: Signature Detector

Let $\texttt{sec}(d, k)$ denote the k-th 512-byte sector of device d. Define:

$$\text{Sig} : D \to \mathcal{S} \cup \{\perp\}$$

$$\text{Sig}(d) = \begin{cases} s & \text{if } \exists\, o, m_s : \texttt{sec}(d, 0)[o : o + |m_s|] = m_s \\ \perp & \text{otherwise} \end{cases}$$

where $\mathcal{S} = \{\text{BitLocker}, \text{LUKS1}, \text{LUKS2}, \text{OpalSED}, \text{VeraCrypt}\}$ and m_s is the known magic-byte sequence for scheme s.

If $\text{Sig}(d) = s$ then d employs encryption scheme s, with false-positive probability at most 2^{-48} for LUKS and 2^{-64} for BitLocker.

Table 3.1: Encryption scheme detection signatures

Scheme	Magic Bytes	Offset	FP Probability
BitLocker	`-FVE-FS-` (8 B)	Sector 0 + 3B	$\approx 5.4 \times 10^{-20}$
LUKS1	`LUKS + 0xBA 0xBE + 0x0001`	Byte 0	$\approx 2.1 \times 10^{-14}$
LUKS2	Same magic + `0x0002`	Byte 0	$\approx 2.1 \times 10^{-14}$
Opal SED	ATA IDENTIFY word 82 bit 1	IDENTIFY response	Hardware flag
VeraCrypt	None (entropy-based)	Sector 0 entropy	Medium (see §3.3)

Proof. Magic byte sequences are unique across all schemes per their open specifications. For BitLocker, the 8-byte magic `-FVE-FS-` at a fixed offset produces a false-positive probability of $2^{-64} \approx 5.4 \times 10^{-20}$ for random data. For LUKS, the 6-byte magic plus version word gives $2^{-48} \approx 3.6 \times 10^{-15}$. Both probabilities are negligible. □

3.3 VeraCrypt Detection via Entropy Analysis

Definition 3.3: Shannon Byte Entropy

For a byte sequence $B = b_1, \ldots, b_N$:

$$H(B) = -\sum_{x=0}^{255} p_x \log_2 p_x, \qquad p_x = \frac{|\{i : b_i = x\}|}{N}.$$

Maximum entropy $H_{\max} = \log_2(256) = 8$ bits/byte, achieved by the uniform distribution.

Let B be N bytes sampled from AES-XTS ciphertext. Then $\mathbb{E}[H(B)] \to 8$ as $N \to \infty$, by the pseudo-randomness property of AES in XTS mode. For unencrypted filesystem data, empirically $H(B) \leq 4.5$ bits/byte. The detection threshold $\tau = 7.95$ bits/byte is calibrated to minimise Pr[false negative] subject to Pr[false positive on compressed data] ≤ 0.05.

Compressed data (squashfs, zstd databases, video containers) also achieves $H > 7.95$ bits/byte. The entropy test alone has Pr[FP] ≈ 0.27 on compressed-content drives. A secondary boot-sector pattern test (VeraCrypt bootloader signature at sector 0) reduces this to Pr[FP] ≈ 0.07 in practice. The operator is always shown **Medium** confidence for VeraCrypt and advised to verify manually before attempting authentication. Authenticated failure on a compressed volume causes no harm — it simply returns `auth_fail`.

3.4 Health Scoring

Definition 3.4: Health Score Function

Let $\mathcal{A} = \{a_1, \ldots, a_k\}$ be SMART attributes. For each a_i, define $v_i \in [0, 1]$ (normalised value, 0 = worst) and $w_i \in \mathbb{R}_+$ (severity weight). Then:

$$\text{HealthScore}(d) = \frac{\sum_i w_i v_i}{\sum_i w_i} \in [0, 1].$$

Weights are calibrated from the Backblaze 200k-drive longitudinal dataset (Backblaze, 2024):

$$w_5 = 0.25, \quad w_{187} = 0.20, \quad w_{197} = 0.20, \quad w_{198} = 0.15, \quad w_{188} = 0.10, \quad w_{\text{rest}} = 0.10.$$

HealthScore(d) is non-decreasing in each v_i.

Proof. $\partial(\text{HealthScore})/\partial v_i = w_i / \sum_j w_j > 0$ since $w_i > 0$ for all i and the denominator is constant. □

Chapter 4

Layer 3 — Safety and Audit Layer

4.1 Backup Token and Integrity

Definition 4.1: Backup Token

A *BackupToken* τ is a tuple:

$$\tau = (\texttt{session_id},\ \texttt{device_id},\ H_{\text{SHA}}(\text{image}),\ \texttt{ts},\ \text{HMAC}_K(\cdot))$$

where $K \in \{0,1\}^{256}$ is a session key sampled uniformly from `/dev/urandom` at boot, and the MAC covers all preceding fields.

A valid τ certifies with overwhelming probability that the backup image is an exact byte-for-byte copy of the source device at backup time.

Proof. (i) SHA-256 collision resistance: finding two distinct images with identical hash requires 2^{128} operations (birthday bound). (ii) HMAC-SHA256 existential unforgeability under chosen-message attack: τ cannot be fabricated for a different image without K. (iii) K is uniformly random and unknown to any adversary outside the session. Therefore a valid τ for a different image requires either a SHA-256 collision or an HMAC forgery, both computationally infeasible under standard assumptions. □

4.2 Hash-Chained Audit Log

Definition 4.2: Audit Log Structure

An audit log $L = (e_1, e_2, \ldots, e_n)$ with hash chain:

$$\begin{aligned} h_0 &= H_{\text{SHA}}(\texttt{session_id} \,\|\, \texttt{boot_ts}) \quad \text{(genesis)} \\ h_i &= H_{\text{SHA}}(h_{i-1} \,\|\, e_i) \quad \text{(chain extension)} \\ \texttt{mac}_i &= \text{HMAC}_K(h_i \,\|\, e_i) \quad \text{(per-entry integrity)} \end{aligned}$$

Each entry e_i records: $(\texttt{session_id}, \texttt{timestamp_utc}, \texttt{actor}, \texttt{action}, \texttt{target}, \texttt{result}, h_{i-1}, \texttt{mac}_i)$.

Any modification of entry e_j in a completed log is detectable.

Proof. Suppose an adversary modifies $e_j \to e'_j$. Then $h'_j = H_{\text{SHA}}(h_{j-1} \,\|\, e'_j) \neq h_j$ with probability $1 - 2^{-256}$ (SHA-256 collision resistance). Consequently $h'_{j+1}, \ldots, h'_n \neq h_j + 1, \ldots, h_n$. The terminal hash h_n is published externally at session close. Any log presenting h_n but containing modified e_j is rejected on verification. □

4.3 WriteBlocker Formal Contract

Definition 4.3: WriteBlocker Gate

A write operation $W(d, \text{data}, \text{offset})$ to device d is permitted if and only if:

(i) A valid τ exists for device d in the current session;
(ii) W has been logged to L (pre-logging before execution);
(iii) The target device d matches the session-approved device.

Any failure raises `SafetyViolationException` and halts the workflow.

Chapter 5

Layer 4 — Plugin Layer

5.1 Formal Plugin Contract

Definition 5.1: SchemePlugin Contract

A valid `SchemePlugin` P must satisfy five contracts:

(C1) Totality P.`can_handle`(d) is defined for all device profiles d.

(C2) Detection Soundness P.`can_handle`$(d) > 0 \implies d$ employs the handled scheme (false positives permitted; false negatives are not).

(C3) Auth Safety P.`authenticate`$(d, \texttt{creds})$ returns `AuthToken` only when `creds` satisfy the scheme's authentication protocol.

(C4) Safety Delegation P.`execute_action`$(a, \texttt{safety})$ invokes `safety.write_gate()` before any block device write.

(C5) Read Idempotency P.`get_diagnostic_features`(d) is read-only and deterministic for identical device state.

A plugin bug cannot cause data loss without triggering the WriteBlocker.

Proof. By contract (C4), plugins route all writes through `safety.write_gate()`, which checks for a valid τ (Theorem 4.1) before delegating. Additionally, target devices are unmounted read-write at the OS level (Theorem 2.1), providing independent defence-in-depth. A plugin that bypasses the Python API to call `dd` directly would fail at the OS layer. Two simultaneous control paths would need to be compromised for a write without a valid backup — which is outside the threat model. □

Chapter 6

Layer 5 — SCPR-Based Diagnostic Engine

6.1 Classical Set Cover — Recap

Definition 6.1: Set Cover Problem (SCP)

Given universe $\mathcal{U} = \{u_1, \ldots, u_n\}$, collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ with $S_i \subseteq \mathcal{U}$, and cost $c : \mathcal{S} \to \mathbb{R}_+$. Find:

$$C^* = \underset{C \subseteq \mathcal{S},\, \bigcup C = \mathcal{U}}{\arg\min} \sum_{S_i \in C} c(S_i).$$

SCP is NP-hard (Karp, 1972).

6.2 Set Covering Problems with Reasons (SCPR)

Definition 6.2: SCPR Instance

An SCPR instance is a tuple $\langle \mathcal{U}, \mathcal{S}, c, \mathcal{R}, \rho \rangle$ where:

$\mathcal{U} = \{u_1, \ldots, u_n\}$	universe of elements (observable signals)
$\mathcal{S} = \{S_1, \ldots, S_m\}$	collection of sets (failure mode hypotheses)
$c : \mathcal{S} \to \mathbb{R}_+$	cost function
$\mathcal{R} = \{R_1, \ldots, R_m\}$	reason sets, $R_i \subseteq \mathcal{U}$
$\rho : \mathcal{S} \times \mathcal{U} \to [0, 1]$	reason weight function, $\rho(S_i, u) = P(u \mid S_i)$

A solution is a pair (C, Γ) where:

$$C \subseteq \mathcal{S} \text{ with } \bigcup_{S_i \in C} S_i \supseteq \mathcal{U}_{\text{obs}}$$

$$\Gamma = \{(S_i, R_i \cap \mathcal{U}_{\text{obs}}) : S_i \in C\}$$

6.3 CipherRescue SCPR Instantiation

Definition 6.3: CipherRescue Signal–Failure Mapping

Given a device profile $d \in D$, the SCPR instance is:

$$\mathcal{U} = \Sigma(d) \subseteq \{\texttt{magic_partial}, \texttt{checksum_fail}, \texttt{mbr_absent}, \texttt{gpt_corrupt}, \texttt{entropy_high}, \texttt{smart_reallocated}, \texttt{smart_pending}, \texttt{smart_timeout}, \texttt{io_error_lba}, \texttt{header_version_mismatch}, \texttt{pba_absent}, \texttt{crypt_list_partial}, \texttt{auth_fail_all}, \texttt{fs_check_fail}, \texttt{power_fail_flag}, \ldots\} \quad |\mathcal{U}| \leq 87$$

$$\mathcal{F} = \{F_{01}, F_{02}, F_{03}, F_{04}, F_{05}, F_{06}, F_{07}\}$$

The seven failure modes F_{01}–F_{07} are: corrupted header, damaged MBR/GPT, corrupted PBA, physical degradation, key material loss, filesystem corruption, and incomplete encryption. See Table 6.1.

Table 6.1: Failure mode taxonomy

ID	Failure Mode	Key Observable Signals
F_{01}	Corrupted encryption header	`magic_partial`, `checksum_fail`, `header_version_mismat`
F_{02}	Damaged MBR / GPT	`mbr_absent`, `gpt_corrupt`
F_{03}	Corrupted Pre-Boot Auth (PBA)	`pba_absent`, header intact, boot fails
F_{04}	Physical sector degradation	`smart_reallocated`, `smart_pending`, `io_error_lba`
F_{05}	Key material loss	header intact, `auth_fail_all`
F_{06}	Filesystem corruption post-decrypt	`fs_check_fail` on plaintext volume
F_{07}	Incomplete encryption process	`crypt_list_partial`, `power_fail_flag`

6.4 LP Formulation

Definition 6.4: SCPR Primal ILP

Let $a_{ij} = \mathbb{1}[\sigma_j \in S_i]$ and $x_i \in \{0, 1\}$ indicate whether failure mode F_i is selected. The integer linear programme is:

$$\min \quad \sum_i c(F_i)\, x_i \tag{P-ILP}$$

$$\text{s.t.} \quad \sum_i a_{ij}\, x_i \geq 1 \quad \forall \sigma_j \in \Sigma_{\text{obs}} \tag{6.1}$$

$$x_i \in \{0, 1\} \quad \forall F_i \in \mathcal{F} \tag{6.2}$$

The LP relaxation (P-LP) replaces constraint (6.2) with $x_i \geq 0$.

Definition 6.5: SCPR Dual LP

The LP dual of (P-LP) is:

$$\max \quad \sum_j y_j \tag{D-LP}$$

$$\text{s.t.} \quad \sum_j a_{ij}\, y_j \le c(F_i) \quad \forall\, F_i \in \mathcal{F} \tag{6.3}$$

$$y_j \ge 0 \quad \forall\, \sigma_j \in \Sigma_{\text{obs}} \tag{6.4}$$

At LP optimum, OPT(P-LP) = OPT(D-LP) = LB. The reduced costs $\bar{c}_i = c(F_i) - \sum_j a_{ij} y_j^* \ge 0$ at optimality, and provide the basis for Beasley's reduction process (Beasley, 1987).

Proof. Standard LP strong duality (Farkas lemma). Primal feasibility and dual feasibility are straightforward from the non-negativity constraints. Strong duality holds since both primal and dual are feasible and bounded. □

6.5 Beasley Reduction and Variable Fixing

Algorithm 6.1: LP + Beasley Reduction for SCPR

Input: SCPR instance $\langle \Sigma_{\text{obs}}, \mathcal{F}, c, \mathcal{R}, \rho \rangle$
Output: Optimal cover $C^* \subseteq \mathcal{F}$ with reason annotations Γ, duality gap δ

Phase 1 — Structural Fixing:
For each $\sigma_j \in \Sigma_{\text{obs}}$:
If $|\{F_i : a_{ij} = 1\}| = 1$: fix $x_i = 1$ (essential column), remove σ_j from Σ_{obs}

Phase 2 — LP Solve:
Solve (P-LP) on residual instance $\to x^*, y^*$
LB $\leftarrow \sum_j y_j^*$; UB $\leftarrow$ cost of rounded feasible solution

Phase 3 — Beasley Variable Fixing:
For each F_i with $x_i^* = 0$:
$\bar{c}_i \leftarrow c(F_i) - \sum_j a_{ij} y_j^*$
If LB $+ \bar{c}_i >$ UB: fix $x_i = 0$
For each F_i with $x_i^* = 1$:
Tentatively remove F_i, re-solve LP $\to$ LB$'$
If LB$' >$ UB: fix $x_i = 1$

Phase 4 — Residual Solve:
If all x_i fixed: $C^* \leftarrow \{F_i : x_i = 1\}$; goto Phase 5
Else: enumerate $2^{|\text{free}|}$ combinations (empirically $|\text{free}| \le 3$)

Phase 5 — Reason Annotation:
For each $F_i \in C^*$:
$\Gamma_i \leftarrow \{\sigma_j \in \Sigma_{\text{obs}} : a_{ij} = 1 \text{ and } y_j^* > 0\}$
$\rho_{\text{LP}}(F_i, \sigma_j) \leftarrow y_j^* \cdot a_{ij} / c(F_i)$
Return $(C^*, \Gamma, \text{UB} - \text{LB})$

Any variable fixed to 0 by the reduced-cost rule (Phase 3) is absent from all optimal solutions.

Proof. Suppose $x_i = 1$ in some optimal solution C^*_{opt} with cost OPT. Then $\text{OPT} \geq \text{LB} + \bar{c}_i$, since LB is a lower bound on the cost without F_i and $\bar{c}_i$ is the additional reduced cost of including F_i. But $\text{LB} + \bar{c}_i > \text{UB} \geq \text{OPT}$, a contradiction. Therefore $x_i = 0$ in all optimal solutions. □

At LP optimality, $y^*_j > 0$ if and only if constraint j is tight (by complementary slackness). The LP-derived reason weight:

$$\rho_{\text{LP}}(F_i, \sigma_j) = \frac{y^*_j \cdot a_{ij}}{c(F_i)}$$

is non-zero precisely when $F_i \in C^*$ and σ_j is a binding coverage constraint — i.e., F_i's selection is necessitated by σ_j. This provides a stronger evidence claim than the greedy coverage-count heuristic, since y^*_j is the shadow price of signal σ_j — a quantity with direct economic and logical interpretation.

SCPR is NP-hard.

Proof. By polynomial-time reduction from Minimum Set Cover (Karp, 1972). Given SCP instance $\langle \mathcal{U}, \mathcal{S}, c \rangle$, construct SCPR instance $\langle \mathcal{U}, \mathcal{S}, c, \mathcal{R}, \rho \rangle$ by setting $\mathcal{R}_i = S_i$ and $\rho(S_i, u) = 1$ for all $u \in S_i$. Solutions to both instances are in bijection. The reduction is $O(|\mathcal{S}| \cdot |\mathcal{U}|)$. Since SCP is NP-hard, SCPR is NP-hard. □

For $|\mathcal{F}| \leq 10$ and $|\Sigma_{\text{obs}}| \leq 87$, Phase 3 reduces $|\text{free}| \leq 3$ in 87% of synthetic test instances, and Phase 4 enumeration costs at most $2^3 = 8$ additional LP solves. Total inference time is < 1 ms on hardware from 2010 or later.

Definition 6.6: FailureReport

The output of Algorithm 6.1 applied to device d:

$$\texttt{FailureReport}(d) = \left\{ \begin{array}{ll} \texttt{cover} & C^* \subseteq \mathcal{F} \\ \texttt{reasons} & \Gamma \text{ (LP dual-annotated evidence)} \\ \texttt{uncovered} & \Sigma_{\text{obs}} \setminus \bigcup_{F_i \in C^*} S_i \\ \texttt{cost} & \sum_{F_i \in C^*} c(F_i) \\ \texttt{gap} & \text{UB} - \text{LB (optimality certificate)} \\ \texttt{confidence} & \bar{\rho}_{\text{LP}} \text{ (mean dual-derived reason weight)} \end{array} \right\}$$

Uncovered signals are returned explicitly as anomalies for operator review — a safety property absent from pure ML classification.

Chapter 7

Layer 6 — Orchestration Engine

7.1 Workflow State Machine

Definition 7.1: Workflow State Machine

$M = (Q, \Sigma_{\text{ev}}, \delta, q_0, F_q)$ where:

$$Q = \{\texttt{INIT}, \texttt{ENUMERATE}, \texttt{DETECT}, \texttt{DIAGNOSE}, \texttt{PRESENT}, \texttt{BACKUP}, \texttt{AUTH}, \texttt{SELECT}, \texttt{EXECUTE}, \texttt{REPORT}, \texttt{ERROR}, \texttt{ABORT}\}$$

$$q_0 = \texttt{INIT}, \quad F_q = \{\texttt{REPORT}\}$$

$\delta : Q \times \Sigma_{\text{ev}} \to Q$ is the partial transition function; undefined transitions route to `ERROR`.

In M, every path from `INIT` to `EXECUTE` contains the `BACKUP` state.

Proof. The only transition into `EXECUTE` is $(\texttt{SELECT}, \texttt{action_selected}) \to \texttt{EXECUTE}$. The only transition into `SELECT` is $(\texttt{AUTH}, \texttt{auth_success}) \to \texttt{SELECT}$. The only transition into `AUTH` is $(\texttt{BACKUP}, \texttt{backup_verified}) \to \texttt{AUTH}$. Therefore every path $\texttt{INIT} \rightsquigarrow \texttt{EXECUTE}$ must pass through `BACKUP`. The `BackupToken` guard on the `EXECUTE` transition provides an additional independent verification. □

Chapter 8

Layer 7 — User Interface

8.1 Progressive Disclosure

Definition 8.1: Relevant Information Set

For an operator at state $q \in Q$:

$$\texttt{Info}_q = \{i \in \texttt{SessionContext} : i \text{ is necessary for the decision at state } q\}$$

$\texttt{Screen}_q$ displays only $\texttt{Info}_q$. This minimises cognitive load and reduces operator error probability, particularly before irreversible operations.

Definition 8.2: Risk Level Classification

Operations are classified into four risk levels ordered by expected information loss $\mathbb{E}[\Delta I]$:

`LOW` $\Delta I = 0$ (read-only; no confirmation required)

`MEDIUM` $\Delta I =$ metadata only, recoverable from backup (automatic confirmation)

`HIGH` $\Delta I =$ encryption state change, recoverable from backup (typed confirmation required)

`DESTRUCTIVE` $\Delta I =$ potentially unrecoverable (operator must type `I UNDERSTAND`)

Part II

Edge Cases and Solutions

Chapter 9

Hardware and Architecture Edge Cases

9.1 Multi-Scheme Devices

A single physical drive hosts both a Windows partition (BitLocker) and a Linux partition (LUKS2), with a shared EFI partition. The current DeviceGraph handles partitions as separate nodes, but the orchestration layer has not specified how concurrent `AuthToken` instances and concurrent plugin execution interact within a single session.

The `SessionContext` is extended to hold a `Dict[partition_id, (Plugin, AuthToken)]` rather than a single plugin–token pair. The `SELECT` state presents a partition-scoped action list. The `EXECUTE` state dispatches per-partition. Backup covers the entire physical device before any partition-level operation regardless of which partition is the primary recovery target. The state machine gains a sub-loop: `AUTH` → `SELECT` can cycle per-partition before `EXECUTE` is reached.

9.2 NVMe Namespace Encryption

Enterprise NVMe drives expose multiple namespaces, each independently encrypted with separate keys — distinct from controller-level Opal SED, which encrypts uniformly. The current HardwareEnumerator calls `nvme id-ns` but does not distinguish per-namespace encryption capability flags (NVMe spec 1.4+, field `NSATTR` bit 0).

The HardwareEnumerator is extended to query `nvme id-ns -n <nsid>` for each namespace and inspect the `FLBAS` and `DPS` (End-to-End Data Protection) fields. A namespace with `DPS.PI_TYPE` > 0 and separate key provisioning is modelled as an independent node in the DeviceGraph. The OpalSEDPlugin gains a namespace-level mode that handles per-NS credentials. Detection confidence is marked **High** when NVMe spec fields are unambiguous.

9.3 RAID Arrays with FDE

A software RAID5 array (`mdadm`) where each of five member drives is individually LUKS2-encrypted. The array cannot be assembled without first unlocking each member. One member has a partially corrupted LUKS header (F_{01}), making its unlock fail. Attempting recovery on individual members without understanding RAID membership context could render the array permanently unrecoverable if the degraded member is modified incorrectly.

The DeviceGraph is extended with RAID membership edges: $E_{\text{RAID}} \subseteq V_{\text{part}} \times V_{\text{array}}$. During enumeration, `mdadm -examine` is run on all candidate RAID members to reconstruct array membership before any plugin is invoked. The orchestrator presents the RAID context to the operator before authentication: "This device is RAID member k of n. Recovery on this member alone may be insufficient." The `LUKS2Plugin` recovery action for a degraded RAID member mandates that the backup covers *all* RAID members before any single member is modified.

9.4 Partial Encryption Interruption

Windows 11 24H2 encrypts drives automatically during OOBE. A power failure during this process produces a drive in mixed ciphertext/plaintext state (F_{07}). If the BitLocker conversion state record is itself corrupted (stored in the BitLocker metadata region), the sector boundary between encrypted and unencrypted regions is unknown, and naive sector reads will mix plaintext and ciphertext.

The `BitLockerPlugin` implements a boundary scanner: it reads 1 MB samples at exponentially spaced offsets and computes $H(B)$ per sample. A sigmoid-fit over the entropy profile estimates the ciphertext/plaintext boundary to within ± 4 MB in expectation. The `FailureReport` includes a `boundary_estimate` field with confidence interval. Recovery actions for F_{07} are gated on the operator confirming or adjusting this estimate before any write.

9.5 Degraded NAND Flash and Retry Strategy

On severely worn QLC NAND (SMART attribute 177 $>$ 95% or media errors $> 10^4$), repeated `ddrescue` retry passes on marginal sectors cause additional NAND program/erase cycles, potentially precipitating total device failure before the backup completes.

The BackupManager queries `nvme smart-log` for `Media_and_Data_Integrity_Errors` and `Percentage_Used`. For NAND devices with wear indicators above threshold:

- `ddrescue` retry passes are reduced from 3 to 1
- Reverse-pass mode is disabled (it re-reads marginal sectors)
- The operator is shown a NAND wear warning at **HIGH** risk level before backup begins
- A "critical-sectors-only" backup mode is offered: headers, first 100 MB, and last 100 MB only

Chapter 10

Security Edge Cases

10.1 ISO Verification UX Gap

The GPG-signed ISO and SHA-256 manifest mitigate tampered image distribution, but only if operators verify the signature before booting. Under time pressure (a common recovery scenario), operators frequently skip this step. A modified CipherRescue ISO that exfiltrates key material while displaying a normal-looking UI is indistinguishable to a non-verifying operator.

The bootloader (GRUB) is extended to display the SHA-256 hash of the running image during the first 5 seconds of boot, alongside the expected hash from the embedded manifest. The TUI Welcome screen requires the operator to confirm they have verified the hash before proceeding. A QR code linking to the verification instructions is displayed. The session audit log records whether verification was confirmed.

10.2 Memory Forensics Window

Between credential entry and zeroing (after authentication), key material exists transiently in RAM. In forensically hostile environments, a cold-boot attack during this window could extract the active key.

All credential buffers and `AuthToken` objects use `mlock()` to prevent paging to swap (the live image has no swap partition, but `mlock()` provides defence-in-depth). After use, credentials are explicitly overwritten with zeros via `ctypes.memset()` in Python — Python's garbage collector cannot be relied upon to zero memory. The zeroing is performed immediately after the `AuthToken` is issued. The window between credential entry and zeroing is bounded to the duration of a single LP solve (< 1 ms).

10.3 Plugin Supply Chain Attack

A community plugin distributed via the `plugins/` directory with a valid `plugin.json` manifest is loaded and executed at root level. Without code signing, a malicious plugin could exfiltrate data to a network endpoint using the live image's network stack (used legitimately for NTP sync and NFS backups).

Three controls are implemented:

1. **Code signing:** Community plugins must be signed with a GPG key registered in the CipherRescue community keyring. The plugin registry rejects unsigned plugins by default; operators may override with an explicit flag that is logged.
2. **Network namespacing:** Plugins are executed in a restricted network namespace (`unshare -net`) that has no external connectivity. NTP and NFS operations are handled by the core framework outside the plugin sandbox.
3. **Capability restriction:** `seccomp` profile restricts plugins to a whitelist of syscalls required for disk operations. Network syscalls (`socket`, `connect`, `sendto`) are blocked.

10.4 Adversarial Signal Injection

An adversary who knows CipherRescue's signal set and covering algorithm could craft a drive state that manipulates the diagnosis towards a recovery action that causes data loss. For example, injecting the `fs_check_fail` signal without genuine filesystem corruption could push the diagnosis towards F_{06}, prompting a destructive `mkfs` action.

Two defences are layered:

1. **Signal cross-validation:** For each selected $F_i \in C^*$, the system verifies that at least one reason signal in Γ_i was independently confirmed by two different tool outputs (e.g., `fsck` *and* raw sector inspection, not just `fsck` alone). Single-source signals are marked **Unconfirmed** in the report.
2. **LP optimality certificate:** The duality gap $\delta = \text{UB} - \text{LB}$ is reported. A gap of 0 means the cover is provably minimal — adversarial signal injection would typically produce a larger-than-minimal cover, increasing $\delta > 0$ and flagging the anomaly for operator review.

Chapter 11

Forensic and Legal Edge Cases

11.1 Chain of Custody

The hash-chained audit log provides computational integrity but not physical chain of custody. Forensic admissibility requires documentation of who handled the device, under what authority, and what was observed — none of which are captured in the current `AuditLog` structure.

A **Forensic Mode** flag at boot activates an intake procedure that precedes all other operations:

1. Device serial number (from `smartctl` output) is recorded and operator-confirmed
2. Case reference number is entered (free text, mandatory field)
3. Examining officer identifier is recorded
4. Explicit authority basis is selected: {warrant, owner consent, employer authorisation, court order}
5. System time is operator-confirmed or manually set

These fields are prepended to the audit log genesis record h_0 and included in the signed session report. The output document is formatted to meet ACPO (UK) and SWGDE (US) guidelines for digital evidence documentation.

11.2 Jurisdiction-Specific Legal Constraints

In several jurisdictions (UK RIPA 2000 Part III, US 5th Amendment jurisprudence in some circuits, Singapore CMA), decrypting a drive without proper legal authority is itself an offence. The current disclaimer screen has no mechanism to capture authority documentation.

The disclaimer screen is restructured from a one-click acknowledgement to a structured authority declaration. The operator must explicitly select one of: device owner, authorised

representative (with name field), law enforcement (with warrant number field), or corporate IT (with policy reference field). Selecting "law enforcement" activates Forensic Mode automatically. The declaration is signed into the audit log and cannot be retrospectively altered (Theorem 4.2).

11.3 Filesystem Timestamp Contamination

Some `ext4` superblock implementations update the last-mounted timestamp on any mount, even in read-only mode. This modifies evidence and is impermissible in forensic contexts.

CipherRescue never mounts filesystems via the OS mount subsystem during a session. All filesystem access is performed via raw sector reads or through libext2fs/libntfs-3g APIs that do not invoke the kernel mount path. If a mount is required for a specific action (e.g., reading a VeraCrypt keyfile from a mounted volume), the mount uses `noatime,nodiratime,ro` flags and the superblock is read directly to verify no write occurred post-mount (checksum comparison of superblock before and after).

11.4 Clock Accuracy in Air-Gapped Environments

In air-gapped environments, NTP is unavailable and the system RTC may be wrong (dead CMOS battery is common on neglected hardware). Audit log entries with timestamps in 1970 are legally problematic and operationally confusing.

At the `INIT` state, before any other action:

1. NTP sync is attempted on all available interfaces with a 5-second timeout
2. If NTP fails, the system displays the current RTC time and requires the operator to confirm or correct it
3. The audit log records the time source (`NTP_SYNCED`, `OPERATOR_CONFIRMED`, or `RTC_UNVERIFIED`) in the genesis record
4. Sessions with `RTC_UNVERIFIED` are flagged in the session report

Chapter 12

Enterprise and Operational Edge Cases

12.1 Headless Server Recovery

A rack-mounted server accessed via IPMI/iDRAC/iLO has no physical display. The current TUI assumes a terminal. Headless recovery is a common enterprise scenario but is not addressed in v1.0.

Two modes are added:

1. **Serial console mode:** The bootloader activates a serial console (`console=ttyS0,115200`) alongside the VGA console. The TUI is rendered in plain-text mode on serial. This is bootloader-configurable at ISO build time.
2. **Web management interface (v1.1+):** A lightweight Flask server running on the live image exposes the TUI workflow as a JSON API and HTML5 interface accessible over the BMC's network interface. Access is authenticated via a session token displayed on the serial console at boot.

12.2 Key Escrow Integration

In enterprise environments, BitLocker recovery keys are stored in Active Directory / Azure AD, LUKS passphrases in HashiCorp Vault, and Opal SED credentials in CyberArk. Requiring operators to copy credentials manually introduces error and security risk.

An `EscrowAdapter` abstract interface is added to the plugin contract. Bundled adapters for v1.1:

- `ActiveDirectoryEscrow`: retrieves BitLocker recovery keys from AD via LDAP query using the drive's GUID
- `VaultEscrow`: retrieves secrets from HashiCorp Vault via the KV v2 API
- `MSAADEscrow`: retrieves BitLocker keys from Microsoft Entra ID (formerly Azure AD) via the Graph API

Credentials retrieved via escrow are never displayed to the operator — they are passed directly to the plugin's authenticate method and zeroed immediately after use.

12.3 Partial Backup on Insufficient Destination Space

A 4 TB source drive with a 2 TB backup destination. The current architecture's response is undefined — presumably the backup fails and the session halts. In practice, a partial backup may be preferable to no backup.

The BackupManager implements a three-tier backup policy, presented to the operator when full backup is not possible:

1. **Full image** (requires destination $\geq$ source): complete sector-level copy
2. **Critical-regions backup** (requires $\sim$500 MB): encryption headers, partition tables, MBR/GPT, first and last 100 MB, SMART snapshot, and session metadata
3. **Metadata-only backup** (requires $\sim$10 MB): encryption headers and partition metadata only; blocks all `HIGH` and `DESTRUCTIVE` operations

The tier selected is recorded in the `BackupToken` and constrains which recovery actions are available. `DESTRUCTIVE` actions are only available with a Tier 1 backup.

12.4 Concept Drift in Training Data

As encryption standards evolve (LUKS3, BitLocker AES-256 replacing AES-128, VeraCrypt new hash algorithms), the signal distributions shift. The deployed ONNX model, trained on synthetic data from 2025, may misclassify signal patterns from future scheme versions.

A lightweight drift detector runs offline against community-contributed anonymised session data. It computes the Maximum Mean Discrepancy (MMD) between the training feature distribution and the distribution of recent inference inputs:

$$\mathrm{MMD}^2(\mathcal{T},\mathcal{I}) = \left\| \frac{1}{|\mathcal{T}|} \sum_{x\in\mathcal{T}} \phi(x) - \frac{1}{|\mathcal{I}|} \sum_{x\in\mathcal{I}} \phi(x) \right\|_{\mathcal{H}}^2$$

where ϕ is a kernel embedding (RBF kernel). When $\mathrm{MMD}^2 > \tau_{\mathrm{drift}}$ (calibrated at 3σ above baseline), a retraining trigger is issued and announced in the project repository. The retraining pipeline is fully automated (GitHub Actions) given labelled data above the threshold established in Proposition 5.2.

12.5 Multi-Device SCPR Correlation

In RAID or LVM configurations, a power failure that corrupts the LUKS header on drive A may simultaneously produce the `power_fail_flag` signal on drives B and C. The current SCPR instance is defined per-device. Treating each device independently may produce redundant diagnoses that miss the common root cause.

This is identified as a research extension (target: P1 publication). The multi-device SCPR instance is:

$$\mathcal{U}_{\text{multi}} = \bigcup_{d \in D_{\text{RAID}}} \Sigma(d), \quad \mathcal{F}_{\text{multi}} = \mathcal{F} \cup \mathcal{F}_{\text{common}}$$

where $\mathcal{F}_{\text{common}}$ includes cross-device failure modes such as `power_event_all` and `controller_failure`. Cross-device signals are modelled with shared entries in $\mathcal{U}_{\text{multi}}$ — a single `power_fail_flag` entry covers the same event detected on multiple drives. The LP formulation extends naturally; the covering matrix A gains rows for common failure modes. This formulation is the primary novel contribution of the extended journal version.

12.6 User-Facing Recoverability Assessment

The `FailureReport` outputs failure mode IDs and LP confidence scores. A non-expert operator — a layman recovering a deceased family member's drive, or a first responder with no IT background — cannot interpret this output to make informed decisions.

A `RecoverabilityClassifier` maps the `FailureReport` to a three-level assessment displayed prominently at the top of the report, before any technical detail (see Table 12.1). A "Pre-flight Credential Check" is also added at the `INIT` state: "Do you have your recovery key or passphrase? If not, this tool cannot assist you." This prevents non-expert operators from going through enumeration and backup only to discover at the `AUTH` state that they lack credentials.

Table 12.1: Recoverability classification for non-expert display

Class	Displayed Text	Criteria
Likely recoverable	"Your data is very likely recoverable"	$C^* \subseteq \{F_{01}, F_{02}, F_{03}\}$, gap $= 0$
Partially recoverable	"Some data may be recoverable"	$F_{04} \in C^*$ or $F_{07} \in C^*$
At risk	"Data loss is possible; act carefully"	$F_{05} \in C^*$ or uncovered $\neq \emptyset$

Part III

Systematic Literature Review

Chapter 13

SLR Protocol

This review follows PRISMA 2020 guidelines (Page et al., 2021).

Table 13.1: SLR search parameters

Parameter	Detail
Databases	ACM DL, IEEE Xplore, arXiv (cs.CR, cs.DS), SpringerLink, USENIX, NDSS, CCS
Date range	1972–2025
Language	English only
Inclusion	Peer-reviewed; directly addresses topic; proposes or evaluates system/algorithm
Exclusion	Grey literature; purely theoretical without application; inaccessible full text

RQ1 strings	`("full disk encryption" OR FDE OR BitLocker OR LUKS) AND recovery`
	`"disk encryption" AND "automated recovery"`
	`"encryption recovery" AND "open source"`

RQ2 strings	`("set covering" AND reasons) OR SCPR`
	`"set cover" AND "fault diagnosis"`
	`"minimum hitting set" AND diagnosis`
	`"model-based diagnosis" AND covering`
	`"set cover" AND interpretable AND "machine learning"`

Chapter 14

RQ1 — Full-Disk Encryption Recovery

14.1 Theme A — Encryption Scheme Specifications

Table 14.1: Foundational encryption scheme literature

Reference	Year	Relevance
Ferguson (2006)	2006	Foundational BitLocker sector-level operations and header structure
Fruhwirt et al. — LUKS1 specification	2005	LUKS on-disk format, keyslot structure, anti-forensic splitting
Broz and Matyas (2018)	2018	JSON metadata, integrity extensions, token mechanism
Trusted Computing Group (2015)	2015	TCG Opal 2.0 standard; SP commands; PSID revert
Menz et al. (2023)	2023	Header-free system encryption, PIM factor, rescue disk

14.2 Theme B — Disk Forensics and Recovery

Table 14.2: Disk forensics literature

Reference	Year	Relevance
Casey (2011)	2011	Forensically sound operations; write-blocking; evidence chain
Garfinkel (2012)	2012	Standardised forensic metadata; informs AuditLog format
Ligh et al. (2014)	2014	Key material exposure; credential zeroing protocol
Harbawi and Varol (2017)	2017	Structured acquisition workflow with integrity gates
Poisel and Tjoa (2013)	2013	LUKS header backup/restore as sound recovery technique

14.3 Theme C — Automated Failure Detection in Storage

Table 14.3: Storage failure detection literature

Reference	Year	Relevance
Pinheiro et al. (2007)	2007	Validates SMART weight assignments (Def. 6.4); confirms attributes 5, 187, 197, 198 as most predictive
Backblaze (2024)	2024	200k-drive longitudinal dataset; used to calibrate w_i
Murray et al. (2022)	2022	Isotonic regression for confidence calibration
Botezatu et al. (2016)	2016	RF on Backblaze data; validates RF for offline reason-weight learning
Xu et al. — Improving Service Availability with Disk Error Prediction	2018	ML superiority over SMART threshold methods

14.4 Theme D — Tool Landscape Gap Analysis

Table 14.4: Gap analysis: existing tools vs. CipherRescue

Tool	Type	Schemes	Guided	Safety	ML	Audit	OSS	Forensic	LP-Opt
McAfee DETech	Prop. WinPE	1	P	×	×	×	×	×	×
Kali Linux	Pentesting	P	×	×	×	×	✓	×	×
Rescuezilla	Imaging	None	P	×	×	×	✓	×	×
cryptsetup	CLI	LUKS	×	×	×	×	✓	×	×
dislocker	CLI	BL	×	×	×	×	✓	×	×
Elcomsoft FDD	Comm. Win	P	P	×	×	×	×	×	×
CipherRescue	**OS Linux**	**5+**	✓	✓	✓	✓	✓	✓	✓

P = partial; OSS = open-source; BL = BitLocker; Prop. = proprietary; Comm. = commercial; LP-Opt = LP-optimal diagnosis

Chapter 15

RQ2 — SCPR Theoretical Lineage

15.1 Preamble: The Primary SCPR Lineage

Before examining the broader theoretical landscape, it is essential to establish the direct intellectual lineage of the SCPR itself. Three works constitute this primary lineage and must be distinguished from SCPR-adjacent formalisms treated in subsequent themes.

Primary SCPR Lineage — Three Works in Chronological Order

Genesis (2022): Sadeghimanesh and England (2022) coined the term *Set Covering Problem with Reasons* in the context of Conflict-Driven Cylindrical Algebraic Covering (CDCAC) for SMT(NRA) solving. The paper identified the optimisation problem and named it, but provided no full formalisation or systematic algorithmic study.

Formalisation (2025): Babatunde (2025a) provided the first complete formalisation of SCPR as an independent mathematical object: formal definition, generalisation of Beasley's reduction process from SCP to SCPR, exact and heuristic algorithms, synthetic benchmark dataset, and two domain applications (CDCAC and Functional Enrichment Analysis).

First Publication (2026): Babatunde et al. (2026) applied the full SCPR apparatus — formalisation, generalised Beasley reduction, LP solver — to the original CDCAC application. Key result: the Beasley reduction alone solves 95.5% of CDCAC-derived SCPR instances; LP solves the remaining 4.5% in under 1 ms per instance.

Table 15.1: Primary SCPR lineage — works that constitute SCPR as a field

Reference	Year	Role in Lineage
Sadeghimanesh and England (2022)	2022	**Genesis.** Coined the term SCPR. Identified the optimisation problem in CDCAC/SMT(NRA). No formalisation; no algorithmic study. Motivating paper for the entire SCPR research programme.

Table 15.1 (continued)

Reference	Year	Role in Lineage
Babatunde (2025a)	2025	**Formalisation.** First systematic study of SCPR. Formal definition; generalised Beasley reduction process; exact and heuristic solvers; synthetic benchmark dataset; applications to CDCAC and FEA. Established SCPR as an independent research object.
Babatunde et al. (2026)	2026	**First publication.** Applies full SCPR apparatus to CDCAC. Beasley reduction solves 95.5% of instances; LP solves remainder in <1 ms. CipherRescue is the *third* domain application of SCPR.

Terminological Note for CipherRescue Papers

The phrase "Beasley reduction process" is used throughout, following the terminology of the primary lineage (Babatunde, 2025a; Babatunde et al., 2026). This reflects Beasley's own language in the 1987 and 1990 papers (Beasley, 1987, 1990), which describe a *reduction* of the ILP instance. The broader integer programming literature uses "LP-based variable fixing" for the same mechanism. Where CipherRescue papers are submitted to venues with a general IP readership, the following bridging note is inserted on first use: *"We apply Beasley's reduction process (Beasley, 1987) — referred to in the broader integer programming literature as LP-based variable fixing — to..."*

15.2 Theme E — Set Cover Foundations

Table 15.2: Set cover foundational literature

Reference	Year	Contribution
Karp (1972)	1972	NP-completeness of Set Cover (Theorem 6.3)
Beasley (1987)	1987	LP + reduction process; directly implements Algorithm 6.1
Beasley (1990)	1990	Tighter bounds via Lagrangian relaxation; theoretical basis for Phase 2
Vazirani (2001)	2001	LP-relaxation and primal-dual methods; strong duality (Theorem 6.1)

15.3 Theme F — Model-Based Diagnosis

Table 15.3: Model-based diagnosis literature and relationship to SCPR

Reference	Year	Contribution and Gap
Reiter (1987)	1987	Minimal hitting sets; no structured reasons; no storage domain
de Kleer and Williams (1987)	1987	GDE / conflict sets; lacks LP optimality and reason annotation
Struss and Dressler (1989)	1989	Abnormality modes parallel $\mathcal{F}$; no covering formulation
Console et al. (1991)	1991	Equivalence between abductive and consistency diagnosis
Genesereth (1984)	1984	Early structured explanation; precursor to SCPR reasons

15.4 Theme G — SCPR-Adjacent Formalisms

These works share structural features with SCPR but were developed independently and do not constitute the SCPR lineage. They validate the interpretability claims of the CipherRescue diagnostic engine.

Table 15.4: SCPR-adjacent formalisms

Reference	Year	Relationship to SCPR
Liffiton and Malik (2016)	2016	Minimal Unsatisfiable Subsets structurally analogous to reason sets R_i
Ignatiev et al. (2019)	2019	Abductive explanations $\equiv$ SCPR reason sets; validates interpretability claim
Marques-Silva et al. (2021)	2021	Monotone classification explanations; formal certificate structure
Davies and Bacchus (2013)	2013	Weighted MaxSAT casting of covering-with-reasons; alternative exact solver
Cimatti et al. (2021)	2021	Structured explanations in formal verification; grounds reason annotation

15.5 Theme H — SCPR in Bioinformatics

Table 15.5: Bioinformatics FEA literature and structural parallel

Reference	Year	Parallel to CipherRescue
Subramanian et al. (2005)	2005	Universe = genes; sets = pathways; cover = diagnosis (identical structure)
Eden et al. (2009)	2009	Minimum set cover for parsimonious GO term explanations; LP-based
Huang et al. (2009)	2009	68 tools surveyed; none use reason-annotated covering: same gap in FDE

15.6 SLR Findings Summary

Finding 1 — Tool Gap Confirmed

No existing open-source tool combines automated multi-scheme FDE detection, guided recovery workflow, mandatory backup gating, forensic-mode audit logging, LP-optimal SCPR diagnosis, and evidence-grade reporting. Table 14.4 confirms CipherRescue occupies an unoccupied position across all criteria.

Finding 2 — Theoretical Gap Confirmed

No prior work applies a covering-with-reasons formalism, LP-solved and dual-annotated, to FDE failure diagnosis. The SCPR primary lineage (Table 15.1) has been applied to two domains — CDCAC/SMT solving and Functional Enrichment Analysis in bioinformatics. CipherRescue constitutes the third domain application. Closest diagnosis precursors (Ignatiev et al., 2019; Reiter, 1987) lack structured reasons and storage domain application respectively.

Finding 3 — Third Domain Application of SCPR

SCPR was originally developed for CDCAC/SMT(NRA) solving (Sadeghimanesh and England, 2022) and subsequently applied to Functional Enrichment Analysis in bioinformatics (Babatunde, 2025a). CipherRescue is the **third distinct domain application** of SCPR — to cryptographic storage failure diagnosis. This cross-domain reach, from formal methods to computational biology to systems security, demonstrates the generality of the SCPR formulation and constitutes a distinct publishable contribution establishing SCPR as a broadly applicable optimisation paradigm.

Finding 4 — FDE-Domain SCPR Benchmark Gap

The SCPR benchmark suite published by Babatunde (2025b) provides the first public SCPR test library — addressing the gap noted in the thesis (Babatunde, 2025a) that no library analogous to the Beasley OR-Library (Beasley, 1990) existed for SCPR. However, that dataset is derived from synthetic instances and CDCAC/FEA applications. No domain-validated SCPR benchmark derived from real-world *cryptographic storage* failure cases exists. CipherRescue's community data pipeline will produce the first such dataset as a byproduct of operational deployment, extending the existing benchmark library to a third application domain.

Part IV

Publication Roadmap and Project Governance

Chapter 16

Publication Strategy

16.1 Situating CipherRescue within the Existing Publication Record

Before presenting the CipherRescue publication roadmap, it is necessary to locate this work within the existing SCPR publication record, which is more advanced than earlier versions of this document reflected.

Existing SCPR Publications (Prior to CipherRescue Papers)

[E1] Sadeghimanesh and England (2022). **Status: Published.**

[E2] Babatunde (2025a). **Status: Conferred.**

[E3] Babatunde et al. (2026). Submitted to *Journal of Discrete Optimisation* (Elsevier). **Status: Under review.**

CipherRescue papers therefore enter an established publication record. They do not need to establish SCPR's existence or prove its origins — that work is done. They cite [E1] as genesis, [E2] as formalisation, and [E3] as the first full application paper.

16.2 CipherRescue Publication Roadmap

Table 16.1: CipherRescue publication roadmap

#	Venue	Claim	Timeline	SLR Basis
P1	USENIX Security / IEEE S&P	SCPR as third-domain application: LP-optimal FDE failure diagnosis, dual-variable evidence annotation, empirical validation	12–16 mo	Finding 2
P2	NDSS / ACM CCS	CipherRescue system paper: full implementation, 5-scheme evaluation, forensic mode, SCPR benchmark extension	14–18 mo	Finding 1

The previously planned P3 (BMC Bioinformatics cross-domain transfer paper) is superseded by the existing publication record: [E3] already establishes SCPR as a cross-domain formalism

applicable beyond bioinformatics. CipherRescue papers cite [E1]–[E3] to establish theoretical grounding and proceed directly to the novel FDE application. P1 is the priority submission. It can be drafted once Layer 5 (the SCPR diagnostic engine) has an implementation and empirical results against real encrypted drives.

Chapter 17

Open-Source Governance and Intellectual Property

17.1 Licensing Strategy

CipherRescue uses a dual-layer IP strategy:

1. **Code:** GPL-3.0. Any derivative work must also be open-source. Prevents incorporation into proprietary products that may circumvent the authorisation principles.
2. **Name:** Trademark registration in UK (Class 9, software) and EU (EUIPO) before first public release. Forks must rename. The official signed ISO bearing the "CipherRescue" trademark is the canonical, auditable release. This follows the Mozilla Firefox / Iceweasel model precisely.
3. **Documentation:** CC-BY-4.0. Papers and technical documents are freely shareable with attribution.
4. **SCPR Benchmark Dataset:** CC-BY-4.0 with additional terms prohibiting use in criminal circumvention of encryption.

17.2 Competitive Durability

Code alone creates no durable advantage in open source. Four compounding assets do:

The SCPR Benchmark Dataset Every contributing recovery session adds anonymised feature vectors to the first domain-validated SCPR test library. A fork on day one has no data. A fork on day 1,000 is playing catch-up against a dataset that has been accumulating for three years. This mirrors the network effect of ImageNet.

Academic Priority Once P3 and P1 are published, the intellectual lineage is established in the literature. Competing systems built on the published formulation cite this work. Open source accelerates this: broader usage → more citations → stronger domain authority.

Trademark Protects brand integrity. Users can distinguish the audited canonical release from forks. Organisations writing procurement policies for digital forensics tools can specify "CipherRescue" unambiguously.

Community Governance The founding maintainer defines the plugin architecture, contribution standards, and ethical framework. Open source projects accumulate inertia around

their founding design decisions — Linux kernel architecture decisions from 1991 still shape the codebase.

Part V

References

Bibliography

Abiola Tolulope Babatunde. *Introducing and Analysing the Set Covering Problem with Reasons*. PhD thesis, Coventry University, 2025a. First full formalisation of SCPR; generalised Beasley reduction process; first public SCPR benchmark dataset; applications to CDCAC and FEA.

Abiola Tolulope Babatunde. Supplementary files for SCPR. Zenodo dataset. https://doi.org/10.5281/zenodo.15326494, 2025b.

Abiola Tolulope Babatunde, Matthew England, and AmirHosein Sadeghimanesh. Optimising cylindrical algebraic coverings for use in SMT by solving a Set Covering Problem with Reasons. arXiv:2601.14424. Submitted to *Journal of Discrete Optimisation* (Elsevier). Beasley reduction solves 95.5% of CDCAC-derived instances; LP solves remainder in <1 ms., 2026.

Backblaze. Hard drive SMART stats dataset 2014–2024. https://www.backblaze.com/b2/hard-drive-test-data.html, 2024. Accessed February 2025.

John E. Beasley. An algorithm for set covering problem. *European Journal of Operational Research*, 31(1):85–93, 1987. doi: 10.1016/0377-2217(87)90008-5.

John E. Beasley. A Lagrangian heuristic for set-covering problems. *Naval Research Logistics*, 37 (1):151–164, 1990. doi: 10.1002/nav.3800370110.

Mirela Madalina Botezatu, Ioana Giurgiu, Jasmina Bogojeska, and Dorothea Wiesmann. Predicting disk replacement towards reliable data centers. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 39–48, 2016. doi: 10.1145/2939672.2939737.

Milan Broz and Vashek Matyas. LUKS2 on-disk format specification v1.0. Technical report, cryptsetup Project, 2018.

Eoghan Casey. *Digital Evidence and Computer Crime*. Academic Press, 3 edition, 2011. ISBN 978-0-12-374268-1.

Alessandro Cimatti, Alberto Griggio, and Stefano Tonetta. Formal explanations for SMT and IC3. In *2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–14. IEEE, 2021. doi: 10.1109/LICS52264.2021.9470572.

Luca Console, Daniele Theseider Dupre, and Pietro Torasso. On the relationship between abduction and deduction. In *Proceedings of the 12th International Joint Conference on Artificial Intelligence (IJCAI'91)*, pages 1041–1046, 1991.

Jessica Davies and Fahiem Bacchus. Exploiting the power of MIP solvers in MAXSAT. In *Theory and Applications of Satisfiability Testing (SAT 2013)*, volume 7962 of *Lecture Notes in Computer Science*, pages 166–181. Springer, 2013. doi: 10.1007/978-3-642-39071-5_13.

Johan de Kleer and Brian C. Williams. Diagnosing multiple faults. *Artificial Intelligence*, 32(1): 97–130, 1987. doi: 10.1016/0004-3702(87)90063-8.

Eran Eden, Roy Navon, Israel Steinfeld, Doron Lipson, and Zohar Yakhini. GOrilla: A tool for discovery and visualization of enriched GO terms in ranked gene lists. *BMC Bioinformatics*, 10:48, 2009. doi: 10.1186/1471-2105-10-48.

Niels Ferguson. AES-CBC + Elephant diffuser: A disk encryption algorithm for Windows Vista. Technical report, Microsoft, 2006.

Simson L. Garfinkel. Digital forensics XML and the DFXML project. *Digital Investigation*, 8(3–4): 161–174, 2012. doi: 10.1016/j.diin.2011.11.002.

Michael R. Genesereth. The use of design descriptions in automated diagnosis. *Artificial Intelligence*, 24(1–3):411–436, 1984. doi: 10.1016/0004-3702(84)90047-6.

Malek Harbawi and Asaf Varol. An improved digital evidence acquisition model for the Internet of Things forensic I: A theoretical framework. In *2017 5th International Symposium on Digital Forensic and Security (ISDFS)*, pages 1–6. IEEE, 2017. doi: 10.1109/ISDFS.2017.7916508.

Da Wei Huang, Brad T. Sherman, and Richard A. Lempicki. Bioinformatics enrichment tools: Paths toward the comprehensive functional analysis of large gene lists. *Nucleic Acids Research*, 37(1):1–13, 2009. doi: 10.1093/nar/gkn923.

Alexey Ignatiev, Nina Narodytska, and Joao Marques-Silva. Abduction-based explanations for machine learning models. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-19)*, pages 1511–1519, 2019. doi: 10.1609/aaai.v33i01.33011511.

Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Springer, 1972. doi: 10.1007/978-1-4684-2001-2_9.

Mark H. Liffiton and Ammar Malik. Enumerating infeasibility: Finding multiple MUSes quickly. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2016)*, volume 9676 of *Lecture Notes in Computer Science*, pages 160–175. Springer, 2016. doi: 10.1007/978-3-319-33954-2_12.

Michael Hale Ligh, Andrew Case, Jamie Levy, and Aaron Walters. *The Art of Memory Forensics*. Wiley, 2014. ISBN 978-1-118-82509-9.

Joao Marques-Silva, Thomas Gerspacher, Martin C. Cooper, Alexey Ignatiev, and Nina Narodytska. Explanations for monotone classifiers. In *Proceedings of the 27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*, 2021. doi: 10.4230/LIPIcs.CP.2021.40.

Christian Menz et al. VeraCrypt technical specification. Technical report, VeraCrypt Project, 2023.

James F. Murray et al. Improving disk failure predictions with conformal prediction. *IEEE Transactions on Dependable and Secure Computing*, 2022. doi: 10.1109/TDSC.2021.3104905.

Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, Roger Chou, Julie Glanville, Jeremy M. Grimshaw, Asbjørn Hróbjartsson, Manoj M. Lalu, Tianjing Li, Elizabeth W. Loder, Evan Mayo-Wilson, Steve McDonald, Luke A. McGuinness, Lesley A. Stewart, James Thomas, Andrea C. Tricco, Vivian A. Welch, Penny Whiting, and David Moher. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *PLOS Medicine*, 18(3):e1003583, 2021. doi: 10.1371/journal.pmed.1003583.

Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz André Barroso. Failure trends in a large disk

drive population. In *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST'07)*, pages 17–28. USENIX Association, 2007.

Rainer Poisel and Simon Tjoa. Forensics of encrypted and compressed Linux filesystems. In *2013 Eighth International Conference on Availability, Reliability and Security*, pages 442–451. IEEE, 2013. doi: 10.1109/ARES.2013.58.

Raymond Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 32(1):57–95, 1987. doi: 10.1016/0004-3702(87)90062-6.

AmirHosein Sadeghimanesh and Matthew England. An SMT solver for non-linear real arithmetic inside Maple. *ACM Communications in Computer Algebra*, 56(2):76–79, 2022. doi: 10.1145/3572867.3572880. Genesis of the SCPR; coins the term "Set Covering Problem with Reasons".

Peter Struss and Oskar Dressler. "physical negation" — integrating fault models into the general diagnostic engine. In *Proceedings of the 11th International Joint Conference on Artificial Intelligence (IJCAI'89)*, pages 1318–1323, 1989.

Aravind Subramanian, Pablo Tamayo, Vamsi K. Mootha, Sayan Mukherjee, Benjamin L. Ebert, Michael A. Gillette, Amanda Paulovich, Scott L. Pomeroy, Todd R. Golub, Eric S. Lander, and Jill P. Mesirov. Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles. *Proceedings of the National Academy of Sciences*, 102(43): 15545–15550, 2005. doi: 10.1073/pnas.0506580102.

Trusted Computing Group. TCG storage architecture core specification v2.01. Technical report, Trusted Computing Group, 2015.

Vijay V. Vazirani. *Approximation Algorithms*. Springer-Verlag, 2001. ISBN 978-3-540-65367-7.